Technical Report for OBD-II InsightBot (Group 18)

Gleb Suchkov Kalmykov (hcwm13), Muhammad Ariz Muhammad Akmal (pgbh35), Kai Chung Matthew Chu (vbzc55), Jaden Zhen Han Yuen (cwms51), Fred Andrew Viner (whhx62)

Date: 10th March 2026

Document Version: 1.0

Table of Contents

1 – Introduction (subtotal 20%) 1.1 - Project Summary (5%) 3

1.2 – Application Set Up (5%) 3

1.3 – Behavioural Requirement Evaluation (10%) 4

2 – Technical Development 5

2.1 – Source Materials 5

2.2 – Technical Description 6

Our Software Development Process 6

Behaviour Driven Development (BDD) Approach 6

Technologies and Platforms used & the Overall System 7

Database Design 8

The AI Engine 11

Voice Service 13

2.3 – Overall Usability 14

3 – Use instructions 14

1. Install Python 15

2. Install Ollama 15

4 – Maintenance and Implications 15

Testing 17

1 – Introduction (subtotal 20%)

1.1 - Project Summary (5%)

With the constant rise of innovative technologies, modern vehicles generate a vast amount of diagnostic data via their OBD-II (On-Board Diagnostic, Version II) systems. A complex system that gives rise to many questions and issues for vehicle owners. We have solved and closed this diagnostic divide, the gap between the complexity of vehicle health information and user comprehension.

IBM commissioned our group to solve this problem utilizing IBM's language model IBM Granite. Our primary client contact throughout the entirety of the project has been John McNamara, Product Owner at IBM. Together, we developed OBD InsightBot: a locally hosted desktop chatbot that allows users to upload their OBD-II logs and have a completely natural conversation, about any problems regarding their vehicle's health.

The InsightBot works by parsing the CSV-formatted OBD-II logs: extracting all key metrics, such as engine RPM, coolant temperature, and throttle position. This data is then fed into a Retrieval-Augmented Generation (RAG) pipeline powered by IBM's Granite 3.3 language model which runs locally via Ollama. Then the InsightBot outputs very clear, understandable and colour coded information regarding the logs. The application also has both speech-to-text dictation and full voice conversation mode, powered by OpenAI's Whisper model, for extra accessibility support. Every response and chat history is stored locally, within the account the user created.

Our main goals for the project were:

- Successfully parse any valid OBD-II CSV and extract all standard metrics
- Provide explanations for all generic diagnostic codes from a database of over 185 codes
- Classify every Granite AI response by severity using a traffic light system, consisting of critical, warning and normal.
- Support both text and voice-based interaction for extra accessibility.

We successfully met every goal that we set during the planning process. Our aim for this project is so that the average person, no matter how much technical literacy they have, can easily navigate our software. We reduced the knowledge gap and exploitation of car owners who fall victim to mechanics taking advantage of them, whilst also helping mechanics

business they would have clients with real car troubles.

The remainder of this report covers the Technical Development of the system (Section 2), User Instructions for Installation and use (Section 3), and Maintenance Considerations including ethical implications (Section 4).

1.2 – Application Set Up (5%)

Username: Tester156
Password: Tester123

1.3 – Behavioural Requirement Evaluation (10%)

BR1 – Account Management				
BR1.1	User creates an account	Fully Met	Unchanged	Accounts are created with validated inputs. Passwords are hashed with bcrypt (12 rounds) and registration is capped at 3 per hour to prevent abuse.
BR1.2	User logs-in to their account	Fully Met	Unchanged	Credentials are verified against stored hashes; a session token is issued on success, and login attempts are limited to 5 per 5-minute window.
BR1.3	User logs out of their account	Fully Met	Unchanged	The session token is invalidated, and the user is returned to the login screen.
BR1.4	User deletes their account	Fully Met	Unchanged	Requires the user to re-enter their password. All associated chats and messages are cascade-deleted.
BR2 – New Chat with Log Upload				
BR2.1	Upload a valid log file	Fully Met	Unchanged	OBDParser validates the CSV structure and extracts 10+ metric types along with any fault codes from a database of over 185 codes.
BR2.2	User uploads an invalid file type	Fully Met	Unchanged	The file extension is checked before parsing begins. Non-CSV uploads receive a descriptive error message.
BR2.3	User uploads a valid file type with bad data	Fully Met	Unchanged	The parser catches missing OBD-II columns and corrupt values, returning specific error messages for each case.
BR2.4	User is logged out and tries to start a chat	Fully Met	Unchanged	Enforced at the UI level: the chat screen is only accessible after a successful login, managed through a QStackedWidget layout.
BR3 – Chat History Management				
BR3.1	User views chat history	Fully Met	Unchanged	Previous chats are listed in the sidebar, ordered by most recent first.
BR3.2	User deletes chat history	Fully Met	Unchanged	Users can delete individual chats or in batches. A confirmation dialog is shown before deletion.
BR3.3	User deletes chat history	Fully Met	Unchanged	Accessible via right-click context menu, with a QInputDialog for inline renaming.
BR3.4	User exports chat log	Fully Met	Modified	Extended to support three export formats: plain text, JSON, and Markdown. The original spec left the format unspecified, so we broadened support.
BR4 – General Vehicle Status Queries				
BR4.1	User asks for a summary when all metrics are at normal level	Fully Met	Unchanged	The RAG pipeline produces a health summary confirming normal readings, citing specific metric values from the log.
BR4.2	User asks for a summary when one or more metrics are abnormal	Fully Met	Unchanged	Out-of-range metrics are flagged with a severity level. The response highlights the issues and offers recommendations.
BR4.3	User asks a question about something that is not contained within the log	Fully Met	Unchanged	If the requested data is not present in the uploaded log, the system says so explicitly.
BR5 – Fault Code Explanation				
BR5.1	User asks to explain a specific, generic fault code	Fully Met	Unchanged	Codes are matched against our 185+ code database and explained through the RAG pipeline, covering description, severity, and likely causes.
BR5.2	User asks for a general summary of all generic fault codes	Fully Met	Unchanged	Every detected fault code is listed alongside its severity and a brief description.
BR5.3	User asks for an explanation of a manufacture specific fault code	Fully Met	Unchanged	Manufacturer-specific codes are recognised but flagged as unsupported, with a note that only generic OBD-II codes can be interpreted.

BR5.4	User asks about fault code when none exist in the log	Fully Met	Unchanged	The system confirms that no faults were found and suggests continuing with routine maintenance.
BR6 – Speech-to-Text Dictation				
BR6.1	User starts and completes basic voice message	Fully Met	Modified	Uses faster whisper (a local Whisper implementation) rather than IBM Granite STT, so that all processing stays offline, and no voice data leaves the machine.
BR6.2	User continues dictation from the correct position	Not Met	—	Dictated text is appended to the end of the input field rather than inserted at the cursor. We chose to focus on getting the core voice pipeline stable first.
BR6.3	User stops speaking and does not press strong	Fully Met	Modified	Recording stops after 2 seconds of silence rather than the originally specified 3. Testing showed that 3 seconds felt sluggish.
BR6.4	Handle microphone permission denied	Fully Met	Unchanged	Microphone availability is checked at startup. If access is denied, the user sees an actionable error message.
BR7 – Voice Conversation Mode				
BR7.1	User starts a voice session and gets a spoken reply	Fully Met	Unchanged	End-to-end pipeline: microphone input is transcribed via faster-whisper, processed by Granite, and read back using piper tts.
BR7.2	User converses with natural turn-taking with endpoints	Fully Met	Modified	Use the same 2-second silence threshold as BR6.3. A half-duplex protocol prevents the microphone from picking up TTS playback, which would otherwise cause feedback loops.
BR7.3	User uses a wake word to activate the chatbot	Not Met	—	Not implemented. A button press is used instead. Wake word detection would need an always-on microphone listener, which conflicts with the privacy-first design and raises resource usage.
BR8 – Danger Level Categorisation				
BR8.1	Information is categorised as critical	Fully Met	Unchanged	Severity Classifier applies keyword analysis, fault code prefix matching (e.g. P03xx misfire codes), and metric thresholds. Critical messages are styled in red with a "Critical" badge.
BR8.2	Information is categorized as potential danger	Fully Met	Unchanged	Warnings are detected through keyword and metric analysis, with negation-aware logic so that phrases like "no critical issue" are not misclassified.
BR8.3	Information is categorised as harmless	Fully Met	Unchanged	Normal responses appear in green. A strong normal signal can override minor warning keywords to avoid false alarms.

Table 1. Evaluation of each Behavioural Requirement created in our initial Requirement Documentation

2 – Technical Development

2.1 – Source Materials

Primarily, the foundation of our system and how we came about our initial development process came from the project brief provided to us by the client. We were given a variety of things to consider, including high level product and service requirements outlining the possible Key Performance Indicators (KPIs) we will be evaluated on. The project brief also provided general expectations of what our product should present.

Other than that, the main drive in ensuring the streamline development of our project came from initial meetings with Mr McNamara, our client from IBM. The office hour sessions we had helped outline the specifications of our project, refine our understandings of what we needed to do and most importantly find out specific requests and features our client wanted us to implement. Gradually, communication with our client became easier through his preferred

platform Discord where any clarifications, updates, and opinions on how the final product should look like became very clear.

2.2 – Technical Description

Our Software Development Process

As previously stated in the Requirements Document, our team adopted a combined development approach of both the Agile and Waterfall methodologies.

We were motivated by the sequential approach that the Waterfall model provided; ensuring we didn't get distracted and stuck to our internal deadlines. This helped streamline our weekly team meetings such that they were focused on a singular goal, allowing complex ideas to be explored and discussed further and in more detail. We were aware of the Waterfall's shortcomings though, hence why we decided to combine with the agile model allowing us to change our goals to align with our clients' vision as and when was needed.

During we development, we made sure that:

1. We defined clear goals and assigned each team member with specific tasks
    - For most of us, this was our first major project, so good planning and open discussions proved to be very important for us.
    - If we were ever unsure of how to approach something, we would get together and discuss before making rash decisions.
2. Followed SCRUM methodologies where appropriate
    - We broke down our development cycle into sprints as described by the Agile development methodology.
    - Each sprint was focused on a specific goal: requirement elicitation; documentation construction; coding the UI, Granite model, etcetera.
    - The final sprints were focused on debugging and refining the final product according to the clients' requests.

Behaviour Driven Development (BDD) Approach

Our team closely followed the principes of BDD throughout the development of our application.

We had a solid foundation of strong, to-the-point, **user stories**. This helped greatly with scope creep as we consistently referred to the behavioural requirements (BRs) defined in our requirements documentation proposed to the client. Since our project motivations remained constant, the system design, coding and everything else followed.

We took advantage of the **MoSCoW method**, a very popular way to prioritise the development of software features. The method categorises goals into 4 categories: must have, should have, could have, won't have. The main benefits of this method include setting clear expectations with the client and within the team; enabling us to stay focused and work on the most important features first; improving our ability to work as a team and make fast decisions and more.

We were fortunate with the fact we didn't need to change our BRs much, other than to slightly change how features were used within the app. These changes were made with the end user in mind, as we tested are application with people in the target demographic who provided feedback on how they would personally use our system.

Technologies and Platforms used & the Overall System

Technology Used	Where we used it	Further comments
Python	Everywhere	Our entire application is made in Python. We don't use the latest version to ensure compatibility with some of the other modules.
Ollama	Backend	The local AI server that hosts the Granite model used. Hosted privately without the need for an internet connection.
IBM Granite 3.3 2B	Backend	The primary Large Language Model (LLM) installed via Ollama. The brains of the chatbot.
Core Python Libraries Used		
LangChain	Backend	Controls most of the AI pipeline. Handles model prompting, routing queries, processing chat history, and instilling context into the LLM inputs.
PyQt6	Frontend/UI	Handles all of UI and its individual components. Highly customisable (in comparison to alternatives like tkinter)
Pandas	Backend	Used for parsing the OBD-II data from the uploaded `.csv` file for further filtering.
Faster-Whisper	Frontend	An optimised version of the Whisper model from OpenAI that enables local Speech-to-Text (STT) conversion.
Piper TTS	Frontend	A local, offline neural text-to-speech (TTS) engine that uses ONNX voice models to synthesize natural-sounding speech.
SQLAlchemy	Backend	Acts as the Object-Relational Mapper (ORM) handling interactions between the application and the local database, i.e., getting user credentials, chat history, etcetera.
Bcrypt	Backend	A module used for the secure hashing of the user passwords.
Pytest	Testing	The main testing framework used for running the unit tests on the application.

Table 2. A table listing the core technologies and libraries used for the application

Our application is complex; we managed this by following object-oriented programming (OOP) techniques, focusing on code reusability and clear use of data structures. Below is a class diagram that helps to visualise how the different parts of our system interact with one another.

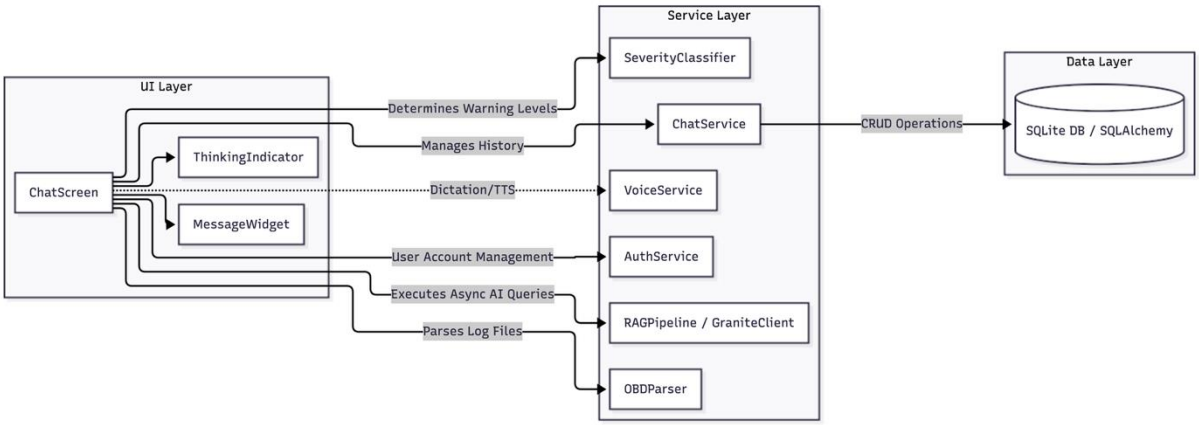

Figure 1. Class Diagram of the core system functionality

Between the team, we were each assigned different components of the system to work on. Then, gradually over time all these individual components were combined to make the final application. Systems that were like one another, for example the `SeverityClassifer` and `ChatService`, tended to be worked on by the same people, to avoid wasting time adjusting to unfamiliar code.

Database Design

User Authentication

Authentication is handled by the `AuthService` class, implementing BR1 regarding account management. The class handles the creation of session tokens on successful login, ensuring chats stay private, as well as more trivial procedures like account deletion. User passwords are hashed using Bcrypt before being stored in the database.

We also apply rate limiting, meaning that the number of times a certain user can attempt to login with their credentials is 5 attempts every 5 minutes. This is to provide some level of deterrence for a brute force attack; though, our system has the advantage of being completely offline.

`AuthService` connects to the local database using SQLAlchemy. Our database consists of three tables: User (authentication, usage), Chat (parsed metrics/fault codes, other metadata) and Message (belongs to a Chat, message severity, message, content, etc). The choice to use a local database instead of one hosted online, was mainly down to the client requesting that the application should be able to run without connection to the internet. This would be something we'd be willing to change, should the client's needs change in the future.

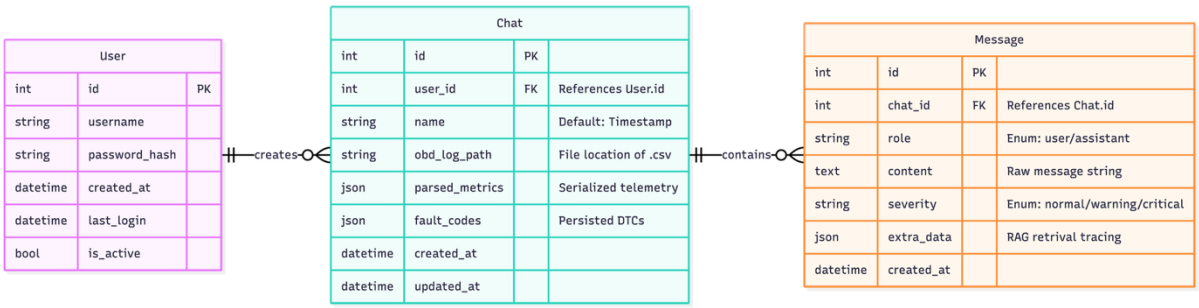

Figure 2. Entity Relationship diagram depicting the three tables (User, Chat, Message) and their attributes

The database is managed by SQLAlchemy; an Object-Relational Mapper (ORM) that sits on top of the base SQLite configuration but provides many benefits in the ways in which we can manage and maintain the system. The main "pull factors" for us were:

- SQLAlchemy's built in `DatabaseSession` class ensures thread-safe database lock conflicts natively. As described above, we spawn multiple threads when handling user requests so this was a must have feature to remove a lot of headaches.
- The backend system can query data freely and then at any time, call the `session.expunge(chat)` method, which completely detaches the Python object from the database session. This means that the UI elements are completely free to read/overwrite/modify the Chat object without a dangerous thread-unsafe connection open to the internal database.
- JSON data is mapped natively to Pythons lists/dicts respectively, which avoids manual dumps that can be error prone. This means that for most of the time, the data is in the right format when we need it.

`ChatScreen` and the Traffic Light System

The `ChatScreen` class handles communication between the main chat user interface, the local database (specifically the Chat and Message tables) and the backend IBM Granite large language model. It can be seen as the "director" of the entire system.

It's core functionality resolves around thread management. We call each thread a `ChatWorker` which inherits from the QThread class provided by PyQt6. This allows us to operate a smooth UI without buffering, whilst simultaneously performing computationally intense routines in the background. Each worker thread can create additional threads when necessary, including those that run the RAG pipeline which pre-processes data before passing it on to the Granite client running within Ollama.

We ensure that UI elements can only be modified by the main thread, so data is shared between them via typed signals using the `pyqtSignal` class from the PyQt6 module. Chat metadata, user information, LLM responses, etcetera, are all shared this way.

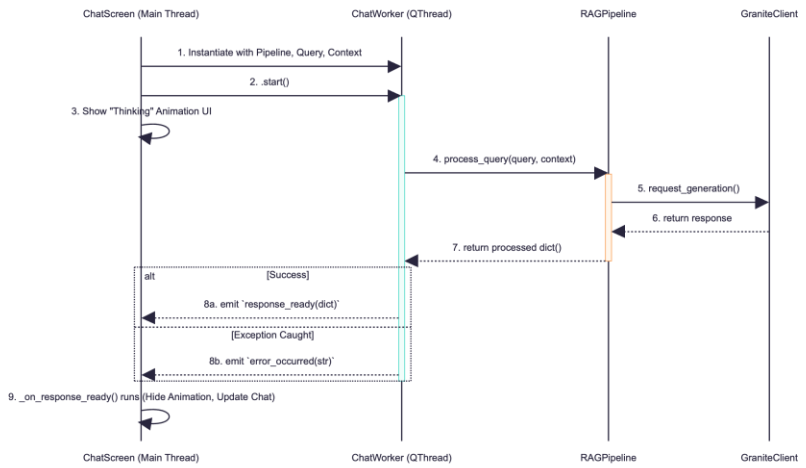

Figure 3. Sequence diagram of thread spawning within the `ChatScreen` class. The green and orange segments depict the lifetime of each of the respective threads.

Chat responses are categorised by severity in accordance with the **Traffic Light System (TLS)**. In one of the first meetings with the client, we discussed how we could go about conveying urgency to the user surrounding possible issues with their vehicle. We landed on the TLS, where every chatbot response is classified as either **Critical**, **Warning**, or **Normal**. Testing feedback we got from potential users reinforced that this type of classification felt intuitive and made responses easier to digest. The `SeverityClassifer` class uses a variety of methods to find the best suited danger level, including word matching, regular expressions and natural language processing techniques.

The AI Engine

The AI system that powers the chatbot is composed primarily of two classes: `RAGPipeline` and `GraniteClient`. They work together to transform raw OBD-II diagnostic data into natural language responses that provide insights into vehicle health.

> **`GraniteClient`** communicates directly with the IBM Granite 3.3 2B model utilising the local Ollama REST API that handles routines like response caching and retry logic.
>
> **`RAGPipeline`** orchestrates the RAG (Retrieval-Augmented Generation) workflow. It receives upstream parse OBD-II data from `OBDParser`, converting it into structured text documents. These documents are stored for retrieval and provided as context that grounds LLM responses in the real vehicle data.

We utilise custom prompt templates in the `PromptTemplates` static class which contains 7 different blueprints depending on the user' request. They're designed to enforce non-technical language, safety-first communication and formatting that's aware of how the severity classification system functions.

Design principles and patterns

We designed and developed this subsystem by:

1. Abstracting the LLM backend behind a single function call `generate_response()`. This was important to us because we didn't want the application to crash if the local Ollama model crashed/stop working. Therefore, we created a demo mode that provides boiler plate responses, which during run-time, we dynamically switch to if the backend cannot connect to the IBM Granite model.
2. Separating the core responsibilities of prompt content, selection and execution into distinct modules. `PromptTemplates` is responsible for the template text, `RAGPipeline` selects the appropriate template to use based on each user message/query, and the `GraniteClient` passes the fully assembled prompt to IBM Granite using the Ollama API.

The RAG pipeline

This component is responsible for improving the output of the LLM by applying pre-processing steps that clearly highlight the relevant information from the user query, uploaded OBD-II data and other contextual information.

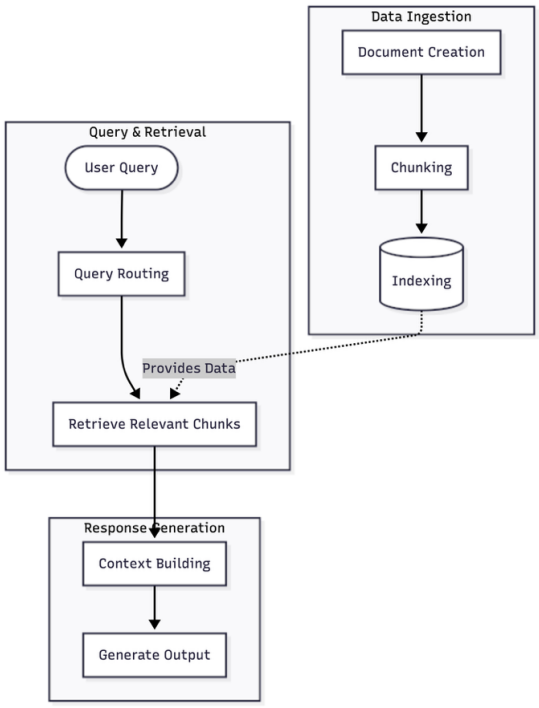

Figure 4. Flow diagram depicting how a chatbot response is generated

Step 1:
The uploaded OBD-II data is parsed into text documents: detailed metric statistics like name, value, unit, natural language description, etc.

Step 2:
The documents are processed using the LangChain module where they are converted into chunks that are stored in a per-chat dictionary.

Step 3:
User queries are assigned an appropriate prompt template based on scanning keywords used. i.e., queries mentioning fault code prefixes are routed to a fault code focused prompt.

Step 4:
A context string is built based on the steps above and injected into the LLM alongside 3 “blocks” containing data about vehicle metrics, fault codes and other relevant information.

The generated output then has some post-processing applied before being pushed to the UI.

Voice Service

There are three core ways to interact with our system: standard texting mode, speech-to-text dictation and voice conversation. These features were implemented based on the corresponding behavioural requirements. For our system to provide a natural, turn-taking conversation experience for the user, we coordinate microphone capture, speech-to-text transcription, AI query processing and then finally text-to-speech playback.

Dictation mode utilises single-segment capture without auto-sending queries to the chatbot.

Voice conversation sees that the user is first listened to, and then the captured voice query is sent automatically. To implement this, we use energy-based voice activity detection since it provides reliable human speech detection by analysing the amplitude of the captured audio.

We designed the processing flow to be highly modular, which helped us to reuse good code for both respective systems wherever possible.

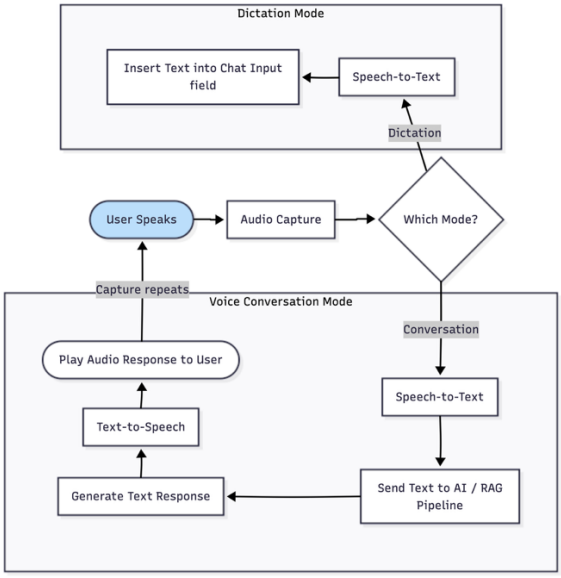

Figure 5. Flow chart depicting how both voice services operate.

2.3 – Overall Usability

Thinking back to our primary aim: creating an approachable application that anyone with no prior expertise can use to receive a comprehensive breakdown of any issues or questions that might arise about their vehicle given their provided OBD-II logs.

We designed our system with this in mind; our application matches the familiar format of other AI apps (Gemini Web App, ChatGPT), so adapting to our specific system should not be to odorous. Buttons use colours with a high contrast in comparison to the other background elements, improving accessibility. Chats are organised cleanly on the left-hand side of the screen, and the main chat window is displayed prominently covering ¾ of the screen space on the right-hand side. This was done consciously so the application never feels too cluttered.

We designed our application with a lack of submenus; once the user has logged in, they have access to almost all the application features within a single (if not two) mouse click. During testing, this was one of the main pieces of positive feedback we received.

When using the voice features, the UI updates clearly when the microphone is in use, and when it is not accepting any input (i.e., when the bot is speaking back to the user in live conversation mode). The buttons to activate these modes have clear emojis to describe their functionality, as well as on-hover tool tips that display to further describe the impact they might have.

3 – Use instructions

The source code for OBD InsightBot is available on GitHub at:
https://github.com/COMP2281/software-engineering-group25-26-18.git

Before installing any of the code, make sure that the machine meets the following requirements:

Necessary-

- Python 3.8 or higher is installed
- Ollama is installed, which can be installed for free from https://ollama.com
- 4 GB of free disk space, not including Ollama

Optional-

- A working microphone, for the voice features

Minimum Specifications-

- 4 core CPU
- 4 GB of RAM

Recommended Specifications-

- 8 core CPU
- 8 GB of RAM
- GPU with CUDA version 12+

Operating Systems-

- Windows 10+
- MacOS version ?

- Linux version ?

1. Install Python

If you already have Python Installed, but are unsure which version, open up command line or terminal and type:

```
# Windows
python --version
# macOS / Linux
python3 --version
```

2. Install Ollama

Download and install Ollama from https://ollama.com, and then pull the Granite model:

```
ollama pull granite3.3:2b
```

Warning: If after installation ollama is not recognised as a command, please restart your terminal/IDE to fix this issue.

Verify everything runs properly with:

```
ollama list
```

Install and run the ap

For Windows (PowerShell)

```
# Extract the zip folder to the working directory (i.e., Desktop)
cd InsightBot
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# Run the app from within the current directory via
python src/main.py
```

For MacOS / Linux

```
# Extract the zip folder to the working directory (i.e., Desktop)
cd InsightBot
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Run the app from within the current directory via
python src/main.py
```

4 – Maintenance and Implications

4.1 – How can the system be maintained?

Due to the offline nature of our developed system, we don't rely that heavily on any external systems. This means it is more than likely our software will work far into the future. However, there are a handful of noteworthy points regarding maintenance:

Run the provided **automated testing** scripts using pytest before every push/pull request to ensure the existing systems are not impacted by the current changes. Use **consistent code formatting** with the aim of improving code clarity for future developers.

Continuously review used **Python dependencies**; replace deprecated packages/methods, monitor the space for security vulnerabilities, update to the latest versions as and when appropriate, etcetera.

Consider monitoring the failure rates and latency within the IBM Granite model with the potential to provide alternative **LLM fallbacks** in case downtime is experienced through Ollama. If newer versions of IBM Granite get added to Ollama, consider **updating the model** to improve overall application performance.

As the demands of the application grow, update the provided **prompt templates** with better constraints and/or formatting expectations.

Write **further system documentation** that would aid future development of the software that other engineering teams could build upon. Adding further detail to function/class docstrings to describe complex logic in an approachable way would help to debug potential problems that might arise later in the software's lifecycle.

4.2 – Future Development

Here are some possible pathways that could be taken to improve upon the application:

Enabling **real-time OBD-II data integration** using a library such as python-OBD that could read live data from a compatible device connected directly to a vehicle's diagnostic port. This would require updating the OBD parser logic to allow for continuous data streaming rather than parsing the data in bulk. This could transform the application from a post-issue diagnostic tool to a live mechanics assistant, or an on-the-road adviser to improve vehicle performance.

Transitioning the UI to **a web application interface** could aid cross-platform compatibility, enable easier system-wide updates without needing for users to download and install new software, and more. This would require a major overhaul to the system. You would need to first extract the core logic found in the app services (like RAG, severity classification, chatting, etcetera) and convert it into a REST API using libraries such as Flask. Then, rebuild the entire frontend UI system as a web app using something like React or Vue (with the option of extending to a mobile application).

Use historical vehicle data to provide **predictive maintenance** information. This would require updating the existing database to a timeseries database (associating time stamps with data) to store the old log files. It would then be possible to train a machine learning model, say logistic regression using a library like scikit-learn, to analyse historical OBD-II metrics that could alert users to problems before an issue such as a check-engine light even appears.

4.3 – Ethical and Societal Impacts

Testing

User acceptance testing

Test Case ID	
Description of test	
Related requirement document details	
Pre-requisites for test	
Test procedure	
Test material used	
Expected result (test oracle)	
Comments	
Created by	
Test environment(s)	

System testing

Integration testing

Unit testing